DELTA FAUCET IDENTIFICATION POC

Technical Architecture, Pipeline Execution Sequence, and Matching Mechanism Guide

Version: 1.0.0 (Production POC)	**Runtime:** Python 3.13 (.venv)	**Model:** OpenCLIP ViT-B-32 (512-dim)
Vector Index: FAISS IndexFlatIP (Cosine)	**Catalog Models:** 297 Delta Models	**Metadata Store:** SQLite + JSON

CRITICAL SYSTEM NOTICE: *Visual candidates only. Confirm the model number before ordering repair parts. Visual similarity does not prove repair-part compatibility. Neural Collaborative Filtering (NCF) is intentionally deferred because no historical user-product interaction data exists.*

1. Complete Execution Sequence — What Runs First

The system is organized into a deterministic seven-stage pipeline. Each stage strictly depends on the verified outputs of preceding stages. The table below outlines the exact command sequence, primary role, and artifact outputs.

Step & Order	Command Executed	Operation & Mechanism	Output Artifacts
Stage 1 Environment	python -m venv .venv pip install -r requirements.txt	Creates isolated Python 3.13 environment; installs PyTorch, OpenCLIP, FAISS-CPU, Pillow, Streamlit, and PyMuPDF.	.venv/ directory with pinned site-packages.
Stage 2 Ingestion	python scripts/extract_catalogue.py	Parses 148-page official Delta PDF catalogue; extracts 297 models and 297 high-res faucet images.	data/products.csv data/image_manifest.csv data/raw/images/*.jpg
Stage 3 Validation	python scripts/validate_catalog.py	Validates schema, checks mandatory keys, verifies disk presence of images, and syncs SQLite store.	data/processed/catalog_validation_report.json data/processed/faucet_catalog.db
Stage 4 Index Build	python scripts/build_index.py	Generates 512-dim OpenCLIP embeddings for all 297 images; builds FAISS IndexFlatIP inner-product index.	data/processed/faucet_index.faiss data/processed/faucet_metadata.json data/processed/index_manifest.json
Stage 5 Automated Tests	python -m pytest -q	Runs 25 automated unit tests across image validation, catalog schemas, hybrid reranking, and retrieval.	Test verification log (25 passed, 0 failures in 17s).
Stage 6 Benchmark	python scripts/evaluate.py	Benchmarks Top-1 accuracy (100%), Top-3 hit rate (100%), MRR (1.0), OOC false accepts (0%), and latency.	data/processed/evaluation_report.json
Stage 7 Interactive UI	python -m streamlit run app/ui.py	Launches Streamlit interface on port 8501. Accepts user photo upload, displays top 3 cards with evidence and disclaimers.	Web UI at http://localhost:8501 with active feedback capture.

2. System Architecture & Information Flow

The diagram below demonstrates how data flows from raw PDF catalog ingestion through vector indexing to real-time query inference.

OFFLINE INGESTION & INDEXING PIPELINE	ONLINE REAL-TIME INFERENCE PIPELINE
1. Raw PDF Catalogue (148 Pages) ↓ *scripts/extract_catalogue.py* **2. Catalog & Images Extracted** • 297 Products (data/products.csv) • 297 Images (data/raw/images/*.jpg) ↓ *scripts/validate_catalog.py* **3. Schema Validation & SQLite Store** • faucet_catalog.db populated ↓ *scripts/build_index.py* **4. OpenCLIP ViT-B-32 Feature Extraction** • 512-dim float32 L2-normalized vectors ↓ *FAISS Inner-Product Build* **5. FAISS Index & Metadata Manifest** • faucet_index.faiss • faucet_metadata.json	**1. Customer Uploads Photo** (UI / File) ↓ *app/image_utils.py* **2. Safe Image Validation & RGB Conversion** • Format (JPEG/PNG/WebP), <10MB limit • EXIF transpose, Decompression bomb safety ↓ *app/embeddings.py* **3. Query Vector Embedding** • 512-dim OpenCLIP vector normalized ↓ *app/retrieval.py* **4. FAISS Vector Search (Top-50 Neighbors)** • Model-level aggregation: max(similarity) ↓ *app/rerank.py* **5. Multi-Signal Hybrid Reranker** • Visual (60%) + Text (20%) + Attr (15%) + Qual (5%) ↓ *app/service.py* **6. Conservative Uncertainty Engine** • likely_match | needs_confirmation | no_match ↓ *app/ui.py* **7. Streamlit Presentation (Top 3 Cards)**

3. Detailed Technical Mechanism Breakdown

This section details the internal mechanics of each algorithmic component in the POC codebase.

Mechanism 1: Safe Image Ingestion & Preprocessing (app/image_utils.py)

Every input image—whether from catalog extraction or customer upload—undergoes strict validation prior to model ingestion to protect against corruption, invalid color profiles, and denial-of-service vulnerabilities:

• **Size & Dimension Guardrails:** Enforces a strict 10 MB maximum file size and pixel dimension checks (50px to 10,000px).

• **Decompression Bomb Protection:** Caps Image.MAX_IMAGE_PIXELS = 50_000_000 to block maliciously crafted decompression payloads.

• **Magic-Byte Format Verification:** Restricts decoding to genuine JPEG, PNG, or WebP images using Pillow byte inspections rather than trusting file extensions.

• **EXIF Orientation Handling:** Automatically applies ImageOps.exif_transpose() to ensure mobile phone photos taken in portrait/landscape are oriented correctly before embedding.

• **Universal RGB Conversion:** Gracefully handles CMYK images (common in print catalogues) and RGBA transparent PNGs by compositing onto a neutral white background to prevent dark border artifacts.

Mechanism 2: OpenCLIP Feature Extraction (app/embeddings.py)

Visual representations are generated using **OpenCLIP ViT-B-32** (pretrained on OpenAI weights). The model runs in inference mode on CPU and is initialized once as a cached singleton outside request loops.

Mathematical Normalization: Raw feature vectors $\mathbf{f} \in \mathbb{R}^{512}$ are L2-normalized:
vec = features / np.linalg.norm(features, axis=-1, keepdims=True)
Because vectors have unit length ($\|\mathbf{u}\|_2 = 1, \|\mathbf{v}\|_2 = 1$), the inner product computed by FAISS is mathematically identical to Cosine Similarity: $$\text{similarity}(\mathbf{u}, \mathbf{v}) = \mathbf{u} \cdot \mathbf{v} = \cos(\theta)$$

Mechanism 3: FAISS Vector Indexing & Inner-Product Search (app/index_store.py)

Vectors are indexed using faiss.IndexFlatIP(512). This provides exact nearest-neighbor search without quantization loss. Parallel metadata maps each 0-indexed vector slot to image ID, model ID, catalog model number, and local image path. Atomic writes (writing to temporary files and renaming) prevent index corruption.

Mechanism 4: Model-Level Candidate Aggregation (app/retrieval.py)

The Problem: A catalog model often has multiple imagery assets (primary view, angled view, handle detail). A standard top-k vector search would allow a single popular model to occupy all top ranks, pushing other viable candidates out.

The Solution: The retriever queries the top 50 nearest vector neighbors and groups results by canonical model_id. It assigns each distinct model the highest score observed among its images: $$\text{ModelScore}(m) = \max_{i \in \text{Images}(m)} \text{CosineSim}(\mathbf{q}, \mathbf{v}_i)$$ Distinct models are then sorted descending by best score, guaranteeing that the customer sees up to three genuinely distinct models.

Mechanism 5: Hybrid Reranking & Weight Renormalization (app/rerank.py)

The initial visual candidates are reranked using a multi-signal composite score combining visual features, optional user description text, verified attributes (category, finish), and catalog completeness:

$$\text{CompositeScore} = w_{\text{img}} S_{\text{img}} + w_{\text{txt}} S_{\text{txt}} + w_{\text{attr}} S_{\text{attr}} + w_{\text{qual}} S_{\text{qual}}$$ **Baseline Weights:** Visual = 0.60, Text = 0.20, Attributes = 0.15, Catalog Quality = 0.05. **Dynamic Weight Renormalization:** If a feature is absent (e.g., customer provides no text description or category filter), the system renormalizes remaining weights to sum to 1.0: $$w'_i = \frac{w_i}{\sum_{j \in \text{Available}} w_j}$$ **Key Design Rules:** (1) Catalog quality measures data completeness, never visual accuracy. (2) Unknown attributes in the catalog are excluded from the attribute evaluation denominator and never penalized as mismatches.

Mechanism 6: Conservative Uncertainty Engine (app/service.py)

The system rejects arbitrary percentage conversions (e.g., claiming a cosine similarity of 0.82 equals '82% probability') and implements strict decision boundaries to protect customers from ordering incompatible replacement parts:

Uncertainty Label	Mathematical Decision Boundary	System Action & Guidance to User
likely_match **HIGH CONFIDENCE**	• Top Score $S_1 \ge 0.85$ • Visual Score $S_{\text{img}} \ge 0.85$ • Score Margin $(S_1 - S_2) \ge 0.03$	High visual confidence with a distinct leader. Presents candidate with model details and prompts user to visually confirm finish.
needs_confirmation **MEDIUM CONFIDENCE**	• Top Score $S_1 \ge 0.80$ • Visual Score $S_{\text{img}} \ge 0.80$ • Score Margin $(S_1 - S_2) < 0.03$	Viable candidate found, but sibling models are visually close. Prompts user to check spout curve, handle count, or provide an additional angle.
no_reliable_match **ABSTENTION**	• Top Visual Score $S_{\text{img}} < 0.80$	System abstains from guessing. Instructs user to capture a clearer photo with proper lighting showing the faucet base, handles, or stamped label.

4. Operational Reference & Verification Results

Metric / Verification Check	Achieved POC Result	Technical Context & Significance
Automated Unit Tests	**25 Passed, 0 Failed** (17.4s)	Full regression suite in pytest -q covering image decode, catalog schema, reranking weights, and candidate deduplication.
In-Catalog Retrieval Accuracy	**100.0% Top-1, 100.0% Top-3**	Mean Reciprocal Rank (MRR) = 1.000 across labeled test queries.
Out-of-Catalog Abstention	**0.00% False Accept Rate**	Non-faucet objects successfully rejected under the 0.80 visual threshold.
CPU Query Latency	**63.2 ms Median (P50)**	Mean latency 275 ms on standard CPU hardware with cached OpenCLIP model.
Delta Catalog Scale	**297 Real Products / Images**	Extracted from official 148-page *Kitchen & Bath Full-Line Catalogue.pdf*.